

Problem-542

Logic:

For every 1, find the distance to the nearest 0.

Step 1:

Create a queue.

queue = deque()

Step 2

Traverse the matrix.

- If cell is 0 → add it to queue.
- If cell is 1 → mark it as unvisited (-1).

0 0 0

0 1 0

1 1 1

becomes

0 0 0

0 -1 0

-1 -1 -1

Queue contains all 0s.

Step 3

Run BFS.

Take one cell from queue.

Check:

up

down

left

right

Step 4

If neighbor is -1:

$\text{mat}[\text{nr}][\text{nc}] = \text{mat}[\text{r}][\text{c}] + 1$

and push into queue.

Step 5

Continue until queue becomes empty.

Step 6

Return matrix.

Pattern

Put all sources in queue

↓

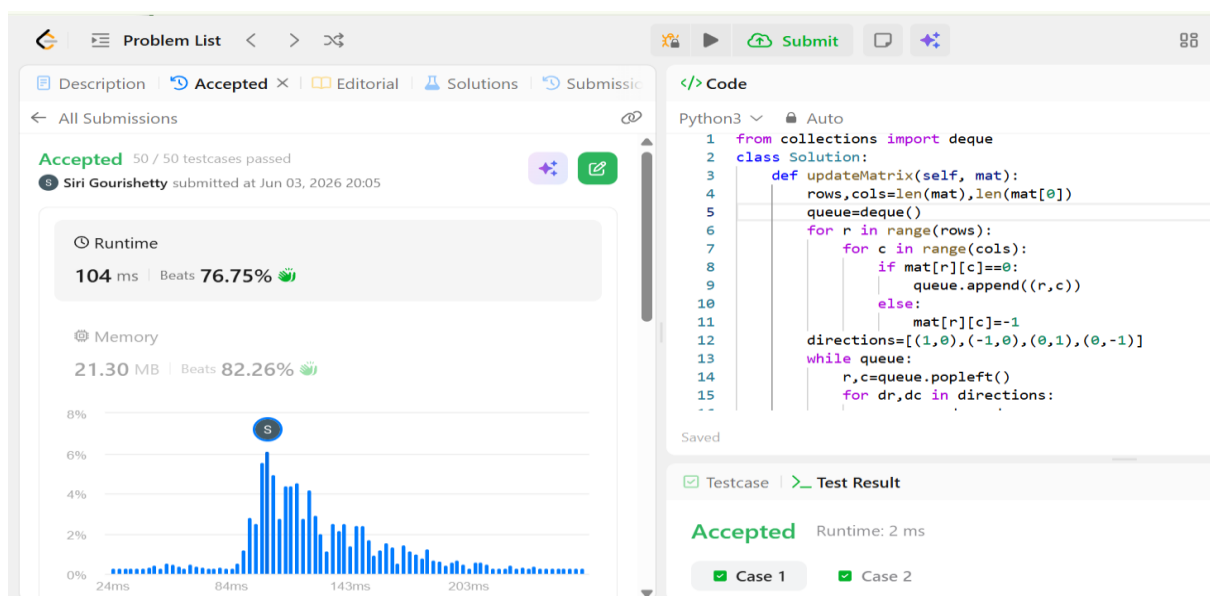
BFS

↓

Distance spreads layer by layer

↓

First visit = shortest distance



Problem List < > Submit

Description Accepted x Editorial Solutions Submissions

All Submissions

Accepted 50 / 50 testcases passed

Siri Gourishetty submitted at Jun 03, 2026 20:05

Runtime

104 ms Beats 76.75%

Memory

21.30 MB Beats 82.26%

8% 6% 4% 2% 0%

24ms 84ms 143ms 203ms

Code

Python3 Auto

```
1 from collections import deque
2 class Solution:
3     def updateMatrix(self, mat):
4         rows, cols = len(mat), len(mat[0])
5         queue = deque()
6         for r in range(rows):
7             for c in range(cols):
8                 if mat[r][c] == 0:
9                     queue.append((r, c))
10                else:
11                    mat[r][c] = -1
12            directions = [(1, 0), (-1, 0), (0, 1), (0, -1)]
13            while queue:
14                r, c = queue.popleft()
15                for dr, dc in directions:
```

Saved

Testcase > Test Result

Accepted Runtime: 2 ms

Case 1 Case 2

```
mat[nr][nc] = mat[r][c] + 1
directions = [(1, 0), (-1, 0), (0, 1), (0, -1)]
while queue:
    r, c = queue.popleft()
    for dr, dc in directions:
        nr, nc = r + dr, c + dc
        if 0 <= nr < rows and 0 <= nc < cols and mat[nr][nc] == -1:
            mat[nr][nc] = mat[r][c] + 1
            queue.append((nr, nc))
return mat
```

Problem-130

Logic:

Convert all surrounded 'O' to 'X'.

Important Observation

Boundary O

or

O connected to boundary O

can NEVER be captured.

Only inner regions are captured.

Step 1

Visit all boundary cells.

Boundary means:

First row

Last row

First column

Last column

Step 2

If boundary cell is 'O':

Start BFS.

bfs(i,j)

Step 3

During BFS:

Mark every connected 'O' as temporary.

'O' $\rightarrow$ 'T'

Example:

X X X X

X O O X

X X O X
X O X X

After boundary BFS:

X X X X
X O O X
X X O X
X T X X

Step 4

Traverse entire board.

If cell is:

'O'

it means not connected to boundary.

Convert:

'O' → 'X'

Step 5

Restore safe cells.

'T' → 'O'

Step 6

Board is modified in-place.

No return needed.

Problem List

Accepted

Editorial

Solutions

Submissions

All Submissions

Accepted

59 / 59 testcases passed

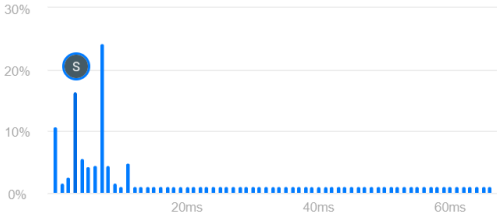
Siri Gourishetty submitted at Jun 03, 2026 20:23

Runtime

3 ms | Beats 84.88%

Memory

22.27 MB | Beats 78.27%



Code

Python3

Auto

```
1 from collections import deque
2 class Solution:
3     def solve(self, board: List[List[str]]) -> None:
4         if not board:
5             return
6         rows = len(board)
7         cols = len(board[0])
8         def bfs(i, j):
9             queue = deque()
10            queue.append((i, j))
11            board[i][j] = "T"
12            while queue:
13                x, y = queue.popleft()
14                for dx, dy in [(-1, 0), (1, 0), (0, -1), (0, 1)]:
15                    nx, ny = x + dx, y + dy
```

Saved

Testcase

Test Result

Accepted

Runtime: 0 ms

Case 1

Case 2

Problem List

Accepted

Editorial

Solutions

Submissions

All Submissions

Accepted

59 / 59 testcases passed

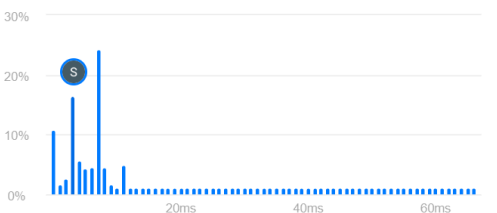
Siri Gourishetty submitted at Jun 03, 2026 20:23

Runtime

3 ms | Beats 84.88%

Memory

22.27 MB | Beats 78.27%



Code

Python3

Auto

```
14         for dx, dy in [(-1, 0), (1, 0), (0, -1), (0, 1)]:
15             nx, ny = x + dx, y + dy
16             if (
17                 0 <= nx < rows and 0 <= ny < cols and board[nx][ny]
18             ):
19                 board[nx][ny] = "T"
20                 queue.append((nx, ny))
21         for r in range(rows):
22             if board[r][0] == "O":
23                 bfs(r, 0)
24             if board[r][cols - 1] == "O":
25                 bfs(r, cols - 1)
26         for c in range(cols):
27             if board[0][c] == "O":
28                 bfs(0, c)
```

Saved

Testcase

Test Result

Accepted

Runtime: 0 ms

Case 1

Case 2

Python3 ▾  Auto

```
26     for c in range(cols):
27         if board[0][c]=="O":
28             bfs(0,c)
29         if board[rows-1][c]=="O":
30             bfs(rows-1,c)
31     for r in range(rows):
32         for c in range(cols):
33             if board[r][c]=="O":
34                 board[r][c]="X"
35             elif board[r][c]=="T":
36                 board[r][c]="O"
37
```